

Digital Design

FF, Register and SRAM Memory with Verilog Model

A. Sahu

Dept of Comp. Sc. & Engg.

Indian Institute of Technology Guwahati

**Some part of Slides Adopted from Digital System Book
by Frank Vahid**

Announcement

- Makeup class on
 - 27th Sept Friday 8-9AM-5G2
 - 04 Oct Friday 8-9AM-5G2

Outline

- Combinational Vs Sequential Logic Design
- Design a **Circuit**, that stores one bit
 - RS latch
- Stabilizing RS latch : Level Sensitive
- Clocked Latch : Flip Flop- Edge Sensitive
- D, JK, T flip flops
- Characterization Table and Equation
 - RS, D, JK and T Flip flop
- Register and Memory

Waveform Simulation: MS-FF

\$iverilog d_ms_ff.v

\$/a.out

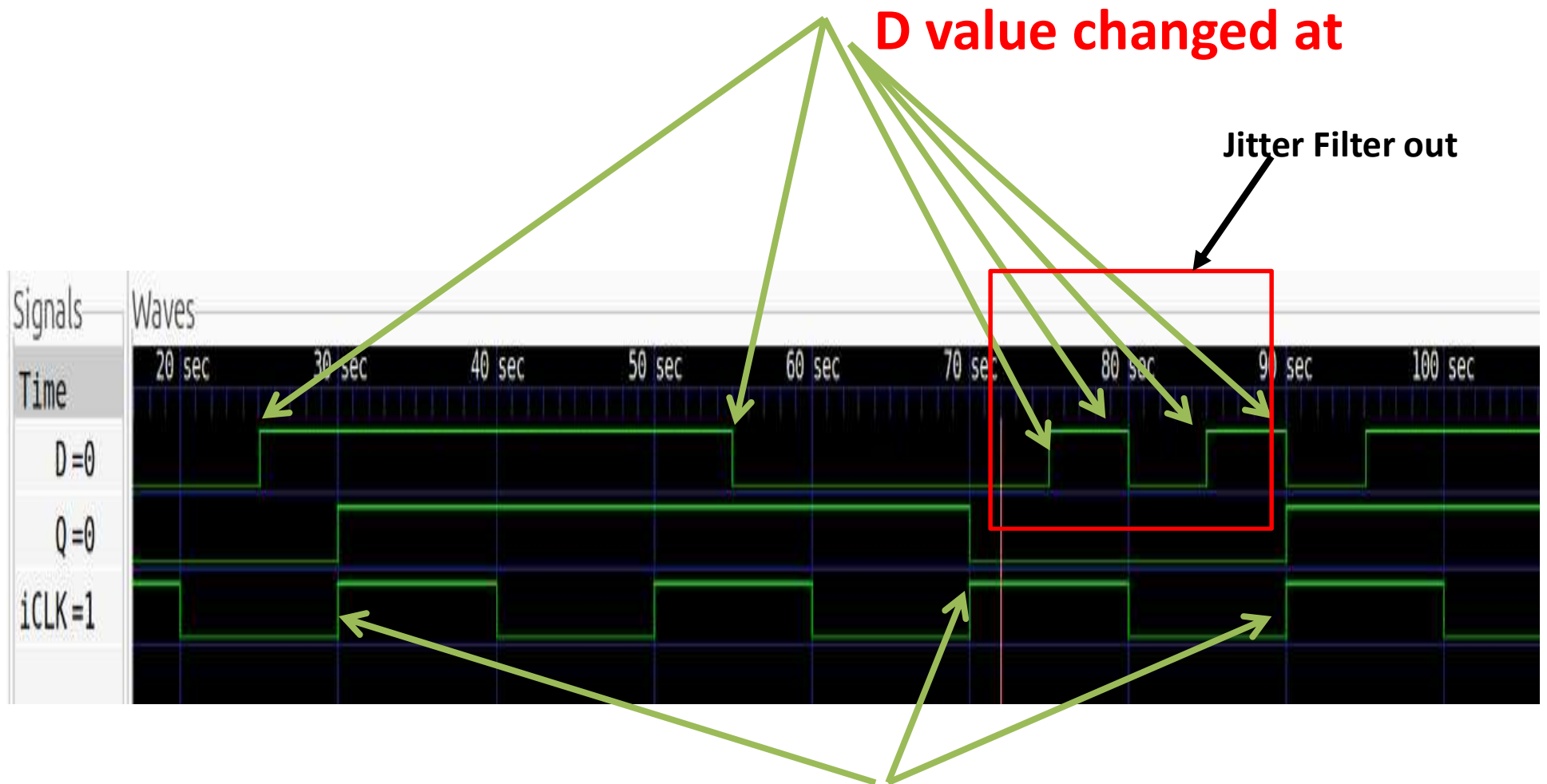
\$gtkwave d_ff_str.vcd

D value changed at



Q values reflected at positive clock edges

Waveform Simulation: MS-FF



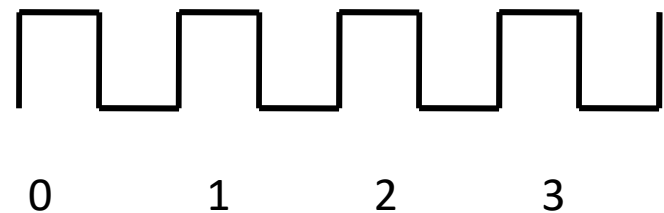
Q values reflected at positive clock edges

Conventions

- The circuit is *set* means output = 1
- The circuit is *reset* means output = 0
- Flip-flops have two output Q and Q'
- Due to time related characteristic of the flip-flop:

– Q_t or Q: present state

– Q_{t+1} or Q^+ : next state

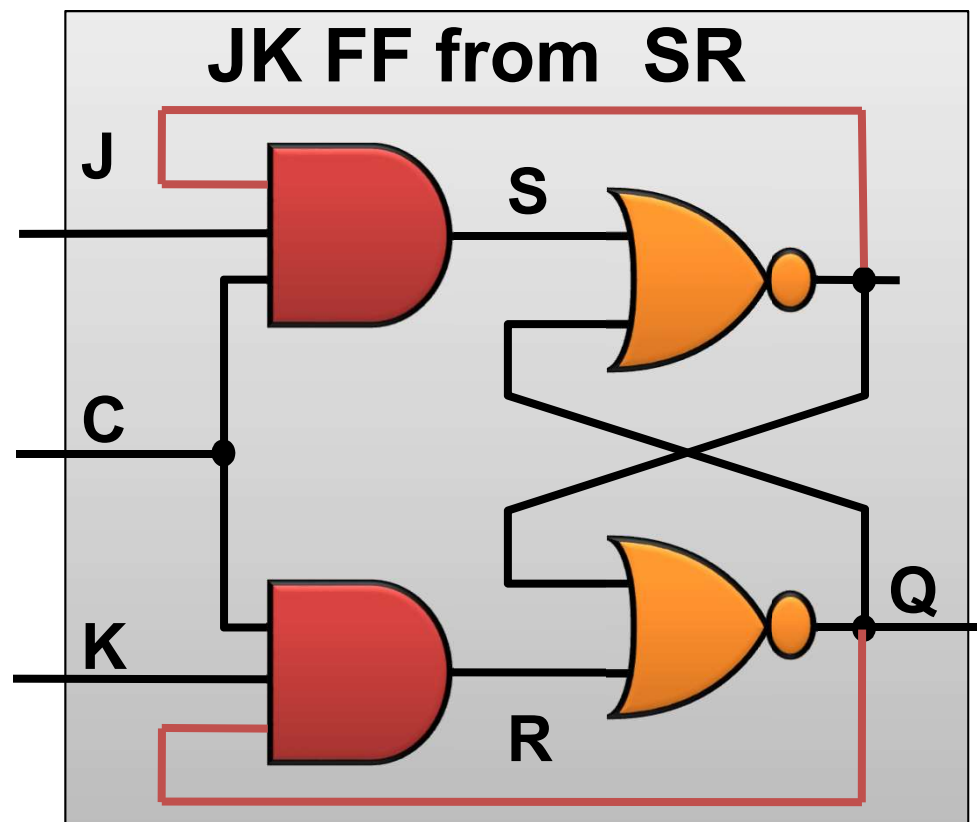


4 Type of Flip Flop

- **SR Flip Flop** : Set/Reset Flip Flop
- **D Flip Flop** : Data Flip Flop to store Bit
- **J-K Flip Flop**: Unavoidable SR=11 state to Toggle (All input values are useful)
 - The JK Flip Flop was named to honour "Jack Kilby" of Texas Instrument engineer who invented the concept of IC.
- **T Flip Flop**: Toggle Flip Flop

J-K Latches

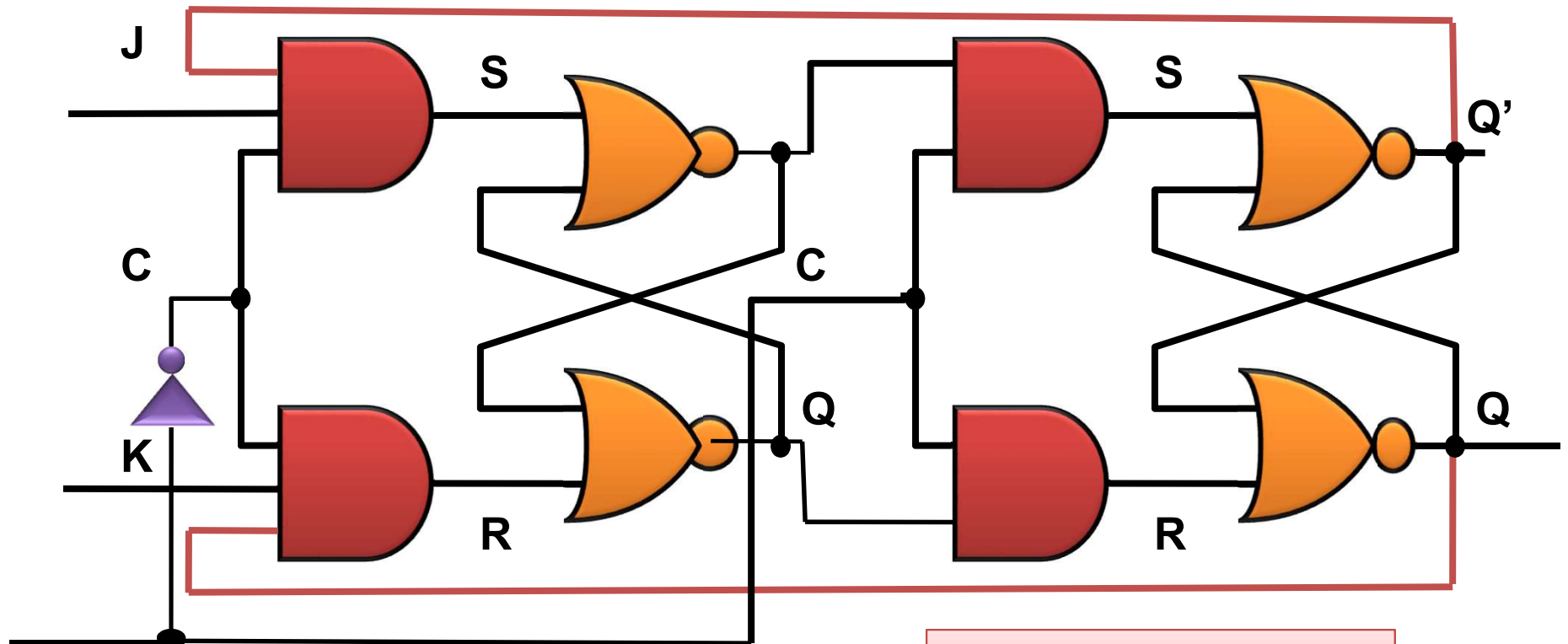
- JK latch** augments the behavior of the SR flip-flop (J=Set, K=Reset) by interpreting the $S = R = 1$ condition as a "flip" or toggle command.



J	K	Q+
0	0	Qt
0	1	0
1	0	1
1	1	Qt'

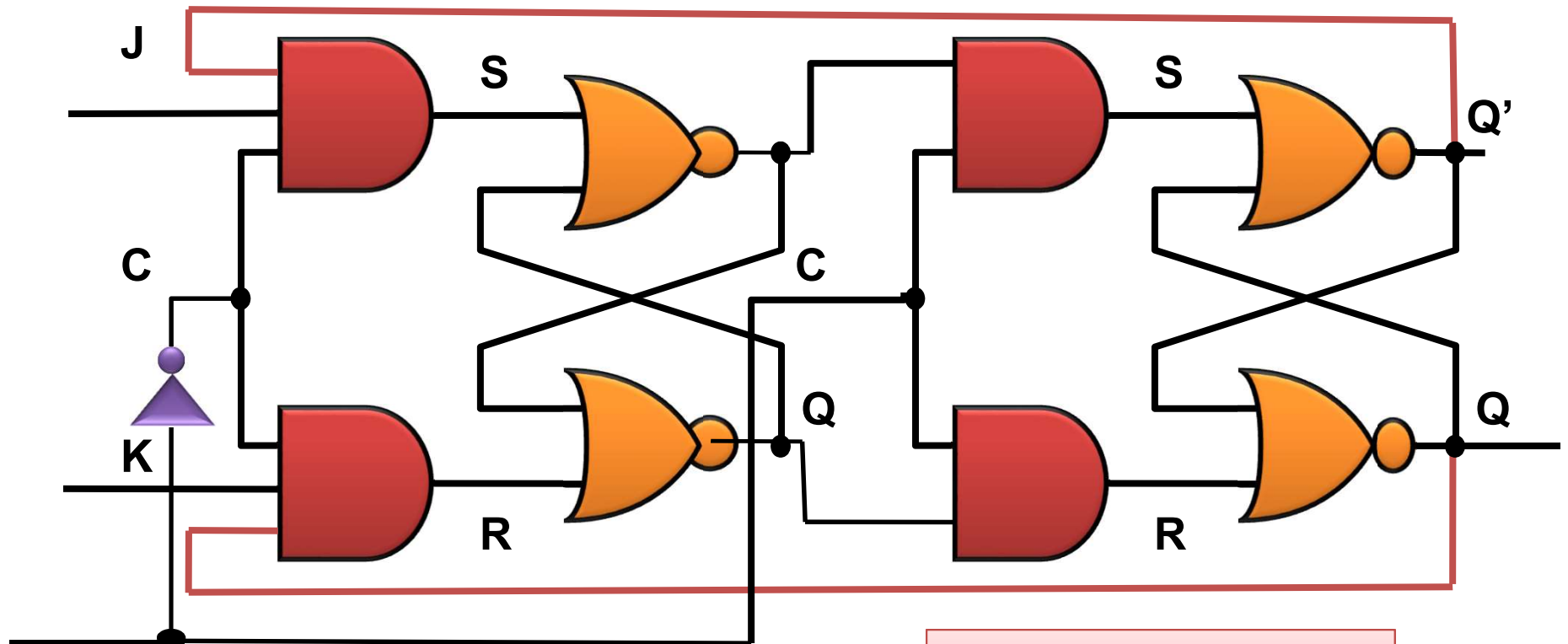
$$Q^+ = K'Q + JQ'$$

Master Slave J-K Flip Flop



$$Q^{+} = K'Q + JQ'$$

Master Slave J-K Flip Flop



$$Q^+ = K'Q + JQ'$$

J-K Flip Flop

- To synthesize a D flip-flop, simply set K equal to the complement of J.
- The JK flip-flop is a universal flip-flop
 - Because it can be configured to work as any FF
 - T flip-flop or D flip-flop or SR flip-flop.

J=T	K=T	Q+
0	0	Q _t
0	1	0
1	0	1
1	1	Q _t '

J=D	K=D'	Q+
0	0	Q _t
0	1	0
1	0	1
1	1	Q _t '

J=S	K=R	Q+
0	0	Q _t
0	1	0
1	0	1
1	1	Q _t '

Verilog Code for JK-FF

```
module jk_ff(input j, input k, input clk, output q);  
  reg q;  
  always @ (posedge clk)  
    case ({j,k})  
      2'b00 : q <= q;  
      2'b01 : q <= 0;  
      2'b10 : q <= 1;  
      2'b11 : q <= ~q;  
    endcase  
endmodule
```

Code for JK-FF

Verilog Code for JK-FF

```
module jk_ff_test;  
reg J K , iCLK;  
wire Q ;  
integer i;  
jj_ff df(J, K, iCLK, Q);
```

```
//Part A : Red part  
//Part B : Orange Part
```

```
endmodule
```

```
initial begin
```

```
iCLK=0;
```

```
for(i=0;i<10; i++) begin
```

```
#10; iCLK = ~iCLK;
```

```
$display("iCLK = %b, J = %b,  
K = %b, Q = %b", iCLK, J, K, Q);
```

```
end
```

```
end
```

```
initial begin
```

```
$dumpfile("jk_ff.vcd");
```

```
$dumpvars;
```

```
J <= 0; K <= 0;
```

```
#5 J <= 0; K <= 1;
```

```
#20 J <= 1; K <= 0;
```

```
#20 J <= 1; K <= 1;
```

```
#20 $finish;
```

```
end
```

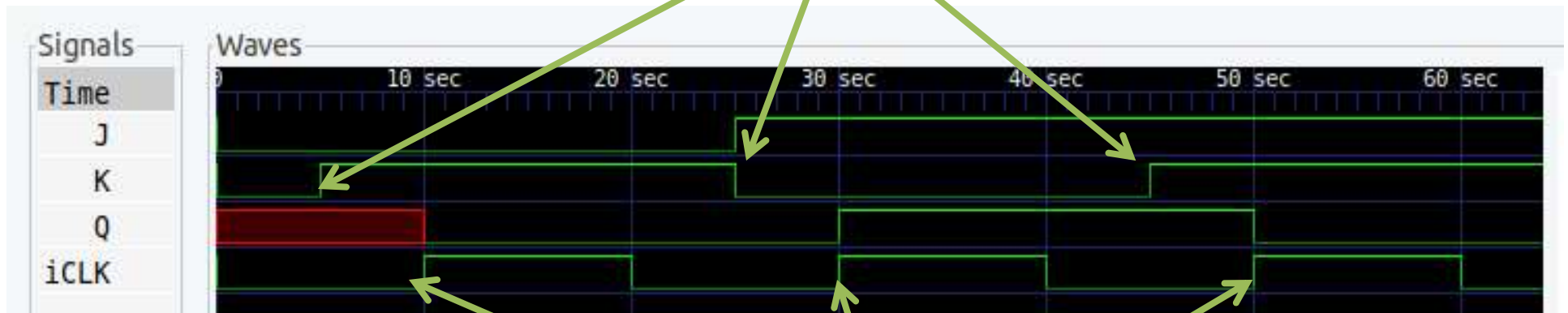
Verilog Code for JK-FF

\$iverilog jk_ff_behav.v

\$/a.out

\$gtkwave jk_ff.vcd

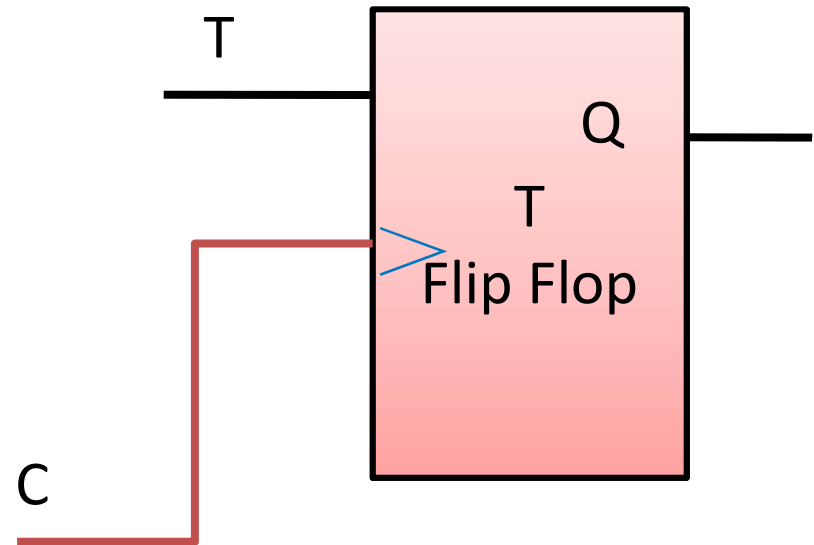
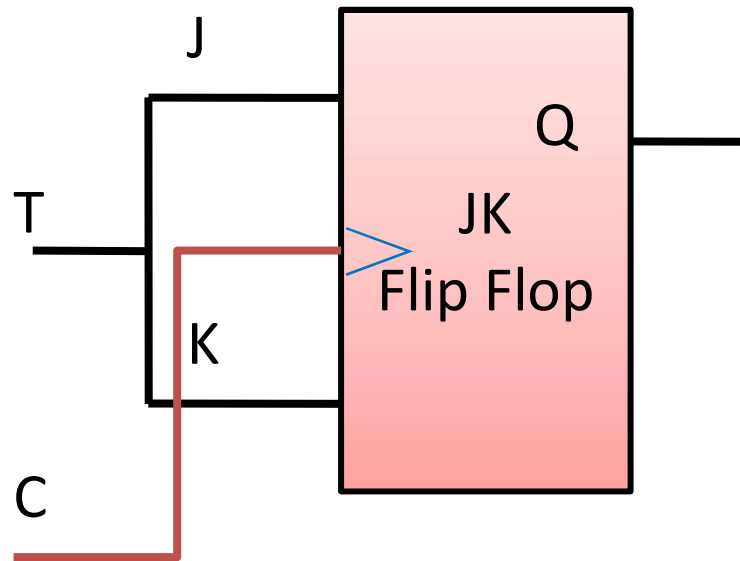
J or K value changed at



Q values reflected at positive clock edges

Toggle Flip-Flop: T FF

- $J=K=1$, $Q^+ = Q'$



Verilog Code for T-FF

```
module t_ff(input t, input clk, output q);  
  reg q;  
  always @ (posedge clk)  
  begin  
    q<=(t ? ~q : q);  
  end  
endmodule
```

Code for T-FF

4 Types of Flip-Flops

S	R	Q+
0	0	Q _t
0	1	0
1	0	1
1	1	U



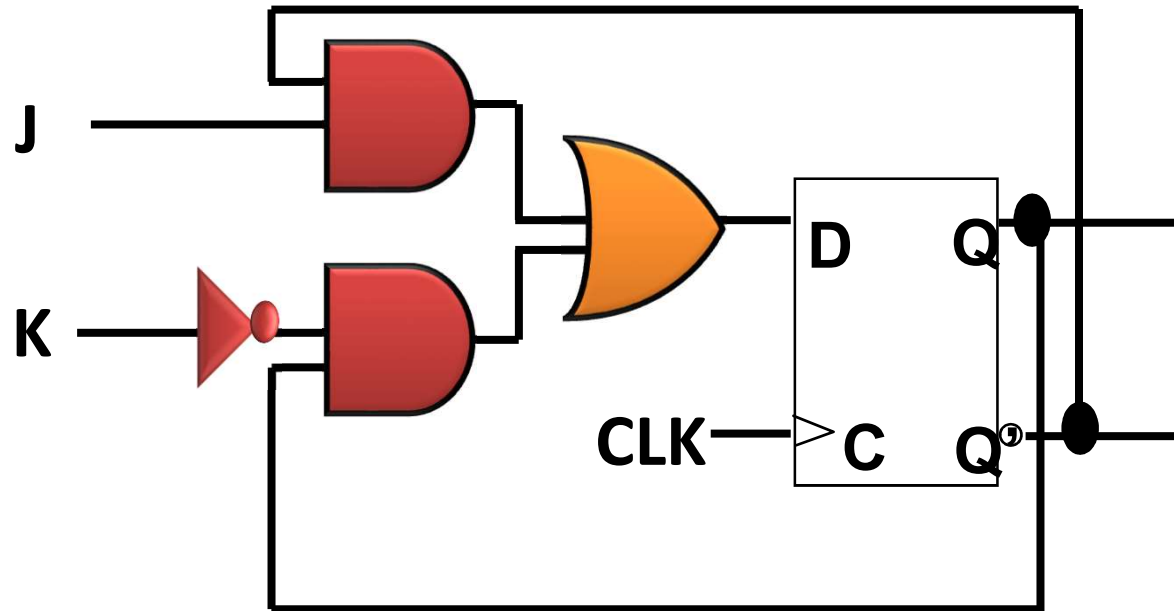
D	Q+
0	0
1	1

J	K	Q+
0	0	Q _t
0	1	0
1	0	1
1	1	Q _t '



T	Q+
0	Q _t
1	Q _t '

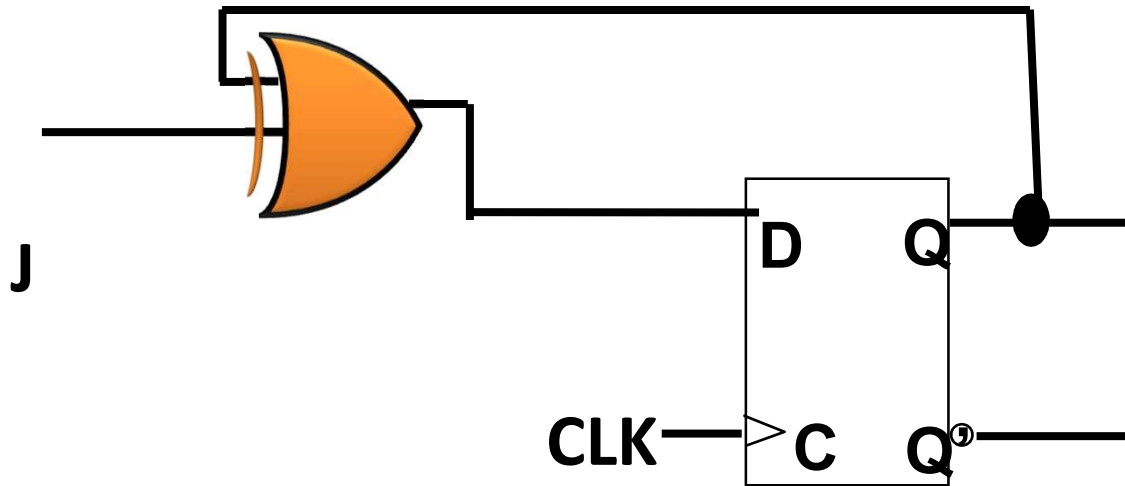
Given a D FF: Construct JK FF



J	K	D	$K'Q + JQ'$
0	0	Q	$1Q + 0Q'$
0	1	0	$0Q + 0Q'$
1	0	1	$1Q + 1Q'$
1	1	Q'	$0Q + 1Q'$

$$D = K'Q + JQ'$$

Given a D FF: Construct T FF



T	D	$TQ' + T'Q$
0	Q	$0Q' + 1Q$
1	Q'	$1Q' + 0Q$

$$D = T Q' + T' Q$$

Characteristic Equations

- A descriptions of the next-state table of a flip-flop
- Constructing from the Karnaugh's map for Q_{t+1} in terms of the present state and input

Characteristic tables

- The tables that we've made so far are called **characteristic tables**.
 - They show the next state $Q(t+1)$ in terms of the current state $Q(t)$ and the inputs.
 - For simplicity, the control input C is not usually listed.
 - Again, these tables don't indicate the positive edge-triggered behavior of the flip-flops that we'll be using.

J	K	Q^+
0	0	Q_t
0	1	0
1	0	1
1	1	Q_t'

D	Q^+
0	0
1	1

T	Q^+
0	Q_t
1	Q_t'

Characteristic equations

- We can also write **characteristic equations**, where the next state $Q(t+1)$ is defined in terms of the current state $Q(t)$ and inputs.

J	K	Q+
0	0	Q_t
0	1	0
1	0	1
1	1	Q_t'

$$\begin{aligned}
 Q^+ &= J'K'Q + JKQ_t' + JK' \\
 &= J'K'Q + JKQ' + JK'Q' + JK'Q \\
 &= J'K'Q + JK'Q + JKQ' + JK'Q' \\
 &= (J+J')K'Q + J(K+K')Q'
 \end{aligned}$$

$$Q^+ = K'Q + JQ'$$

$$Q(t+1) = K'Q(t) + JQ'(t)$$

Characteristic equations

- We can also write **characteristic equations**, where the next state $Q(t+1)$ is defined in terms of the current state $Q(t)$ and inputs.

D	Q^+
0	0
1	1

$$Q^+ = D$$

$$Q(t+1) = D$$

T	Q^+
0	Q_t
1	Q_t'

$$Q^+ = T'Q + TQ' = T \oplus Q$$

$$Q(t+1) = T'Q(t) + TQ'(t) = T \oplus Q(t)$$

Characteristic equations

	SR			
	00	01	11	10
0	0	0	-	1
1	1	0	-	1

$$Q^+ = S + R'Q$$

Flip Flop Type	Characteristic Equation
SR	$Q^+ = S + R'Q$
JK	$Q^+ = JQ' + K'Q$
D	$Q^+ = D$
T	$Q^+ = TQ' + T'Q = T \oplus Q$

FF with Asynchronous Inputs

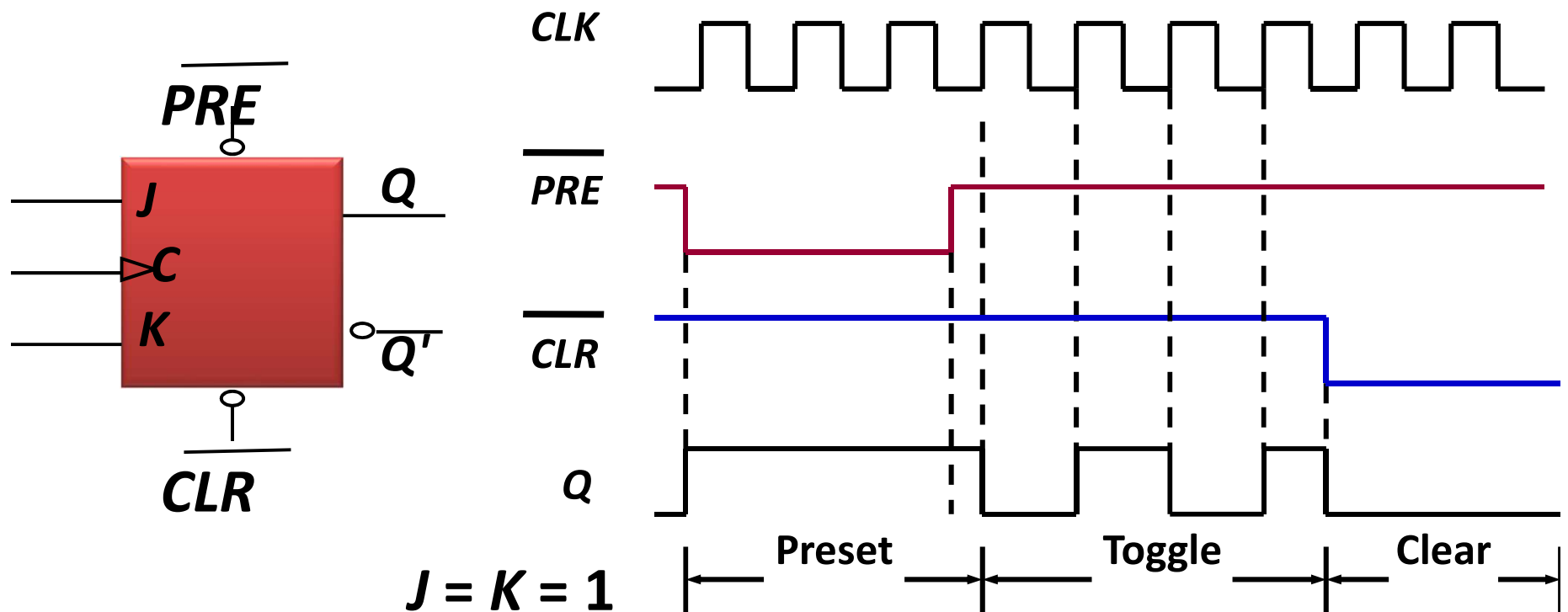
- S-R, D and J-K inputs are synchronous inputs
 - As data on these inputs are transferred to the flip-flop's output
 - Only on the triggered edge of the clock pulse.
- **Asynchronous** inputs affect the state of the flip-flop independent of the clock
- Example:
 - *Preset (PRE) and clear (CLR)*
 - or *direct set (SD) and direct reset (RD)*

FF with Asynchronous Inputs

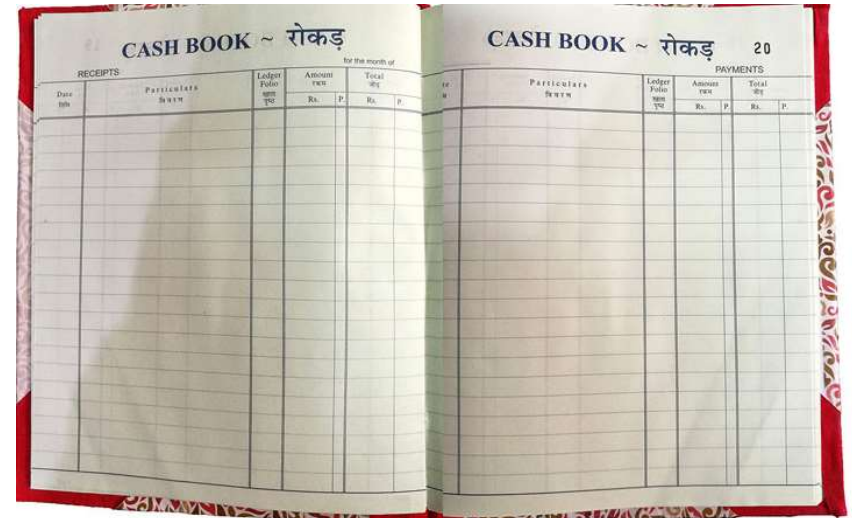
- When $PRE=HIGH$
 - Q is immediately set to HIGH.
- When $CLR=HIGH$
 - Q is immediately cleared to LOW.
- When both PRE and CLR are LOW.
 - Flip-flop in normal operation mode

Asynchronous Inputs

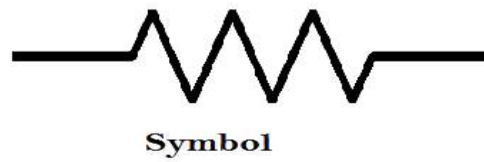
A J-K flip-flop with active-LOW preset and clear inputs



Register



not Resistor



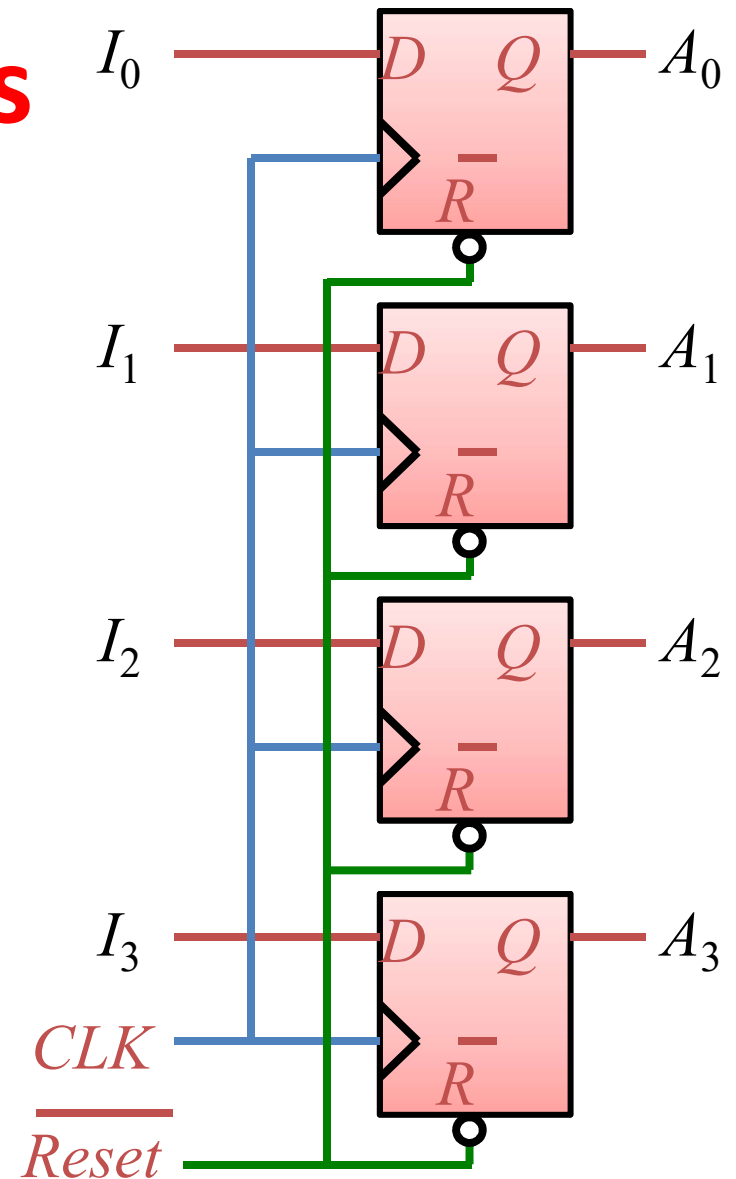
Symbol



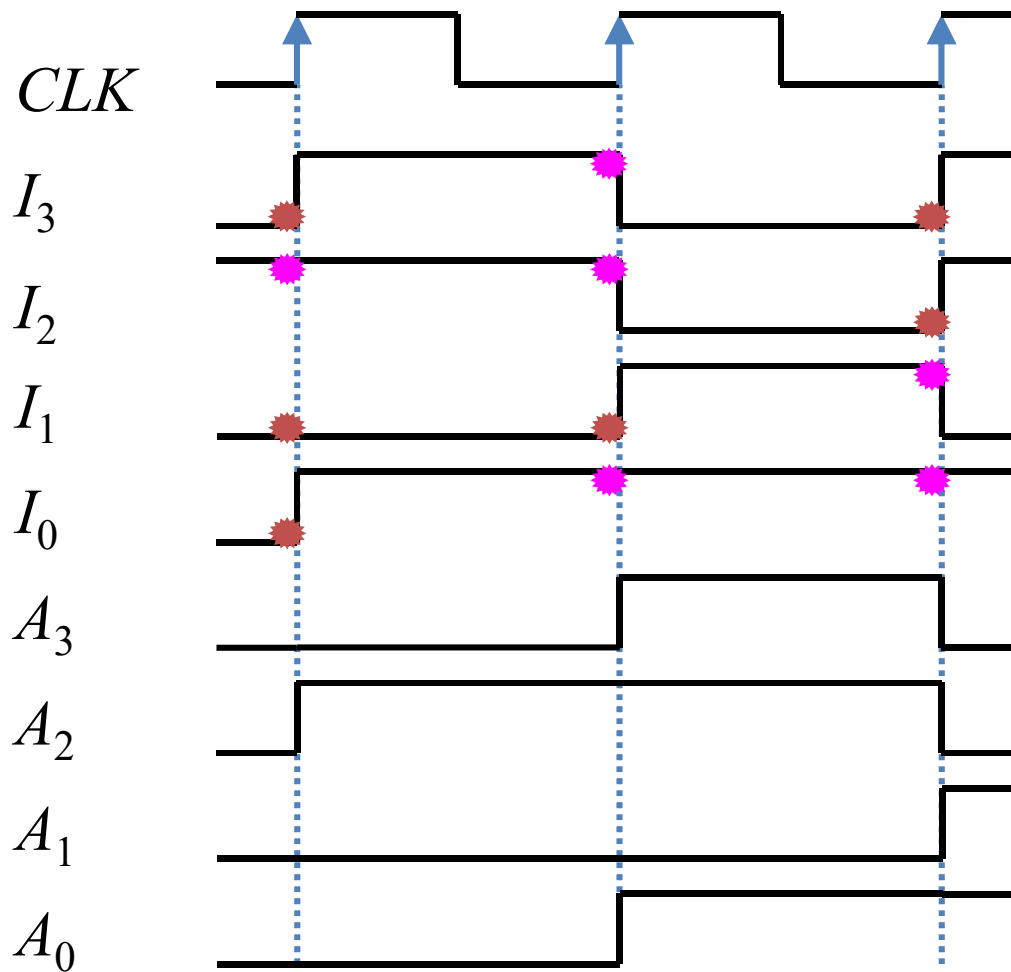
Resistor

Registers

- Group of D Flip-Flops
- Synchronized (Single Clock)
- Store Data

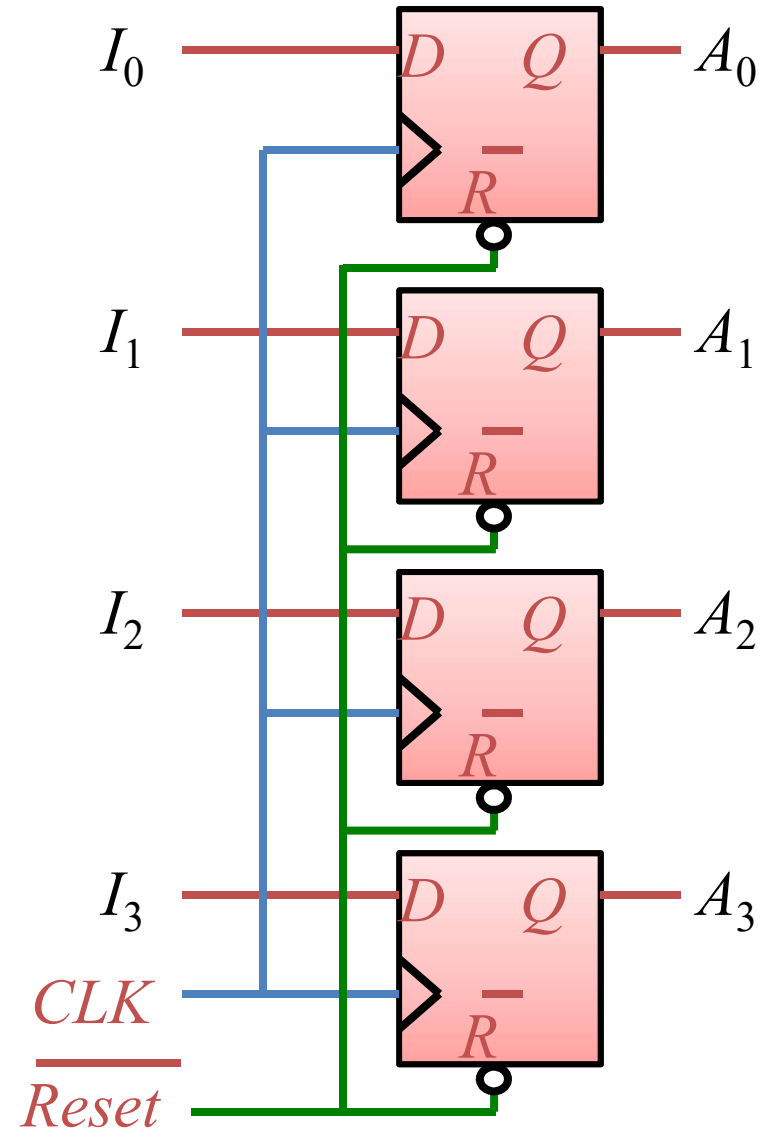


Registers



Note: New data has to go in with every clock

**See carefully: Input at the dotted will be reflected to output :
Just before rising edges**



Verilog Code for Register

```
module Register(clock, r_enable, data_in, data_out);  
input      clock;  
input      r_enable;  
input      [3:0] data_in;  
output reg [3:0] data_out;  
reg [3:0] data_stored;
```

```
always @(posedge clock) begin  
    if(r_enable)    data_stored = data_in;  
end
```

```
always @(posedge clock) begin  
    data_out <= data_stored;  
end
```

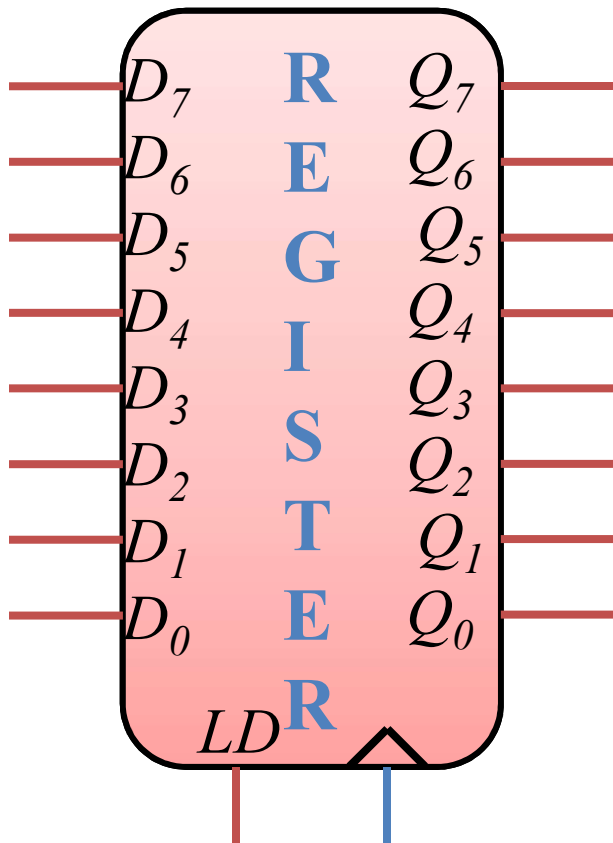
```
endmodule
```

Verilog Code for Register

```
module RegisterTestbench;  
reg      clock = 0; reg      enable = 1; reg [3:0] value_in;  
wire [3:0] value_out;  
always #5 clock = !clock;  
initial $dumpfile("registertestbench.vcd");  
initial $dumpvars(0, RegisterTestbench);  
    Register r(clock, enable, value_in, value_out);  
initial begin  
    # 5 value_in = 3;  
    # 5 value_in = 12;  
    # 5 value_in = 5;  
    # 5 value_in = 14;  
    # 5 value_in = 1;  
    # 5 $finish;  
end  
endmodule
```


Registers with Parallel Load

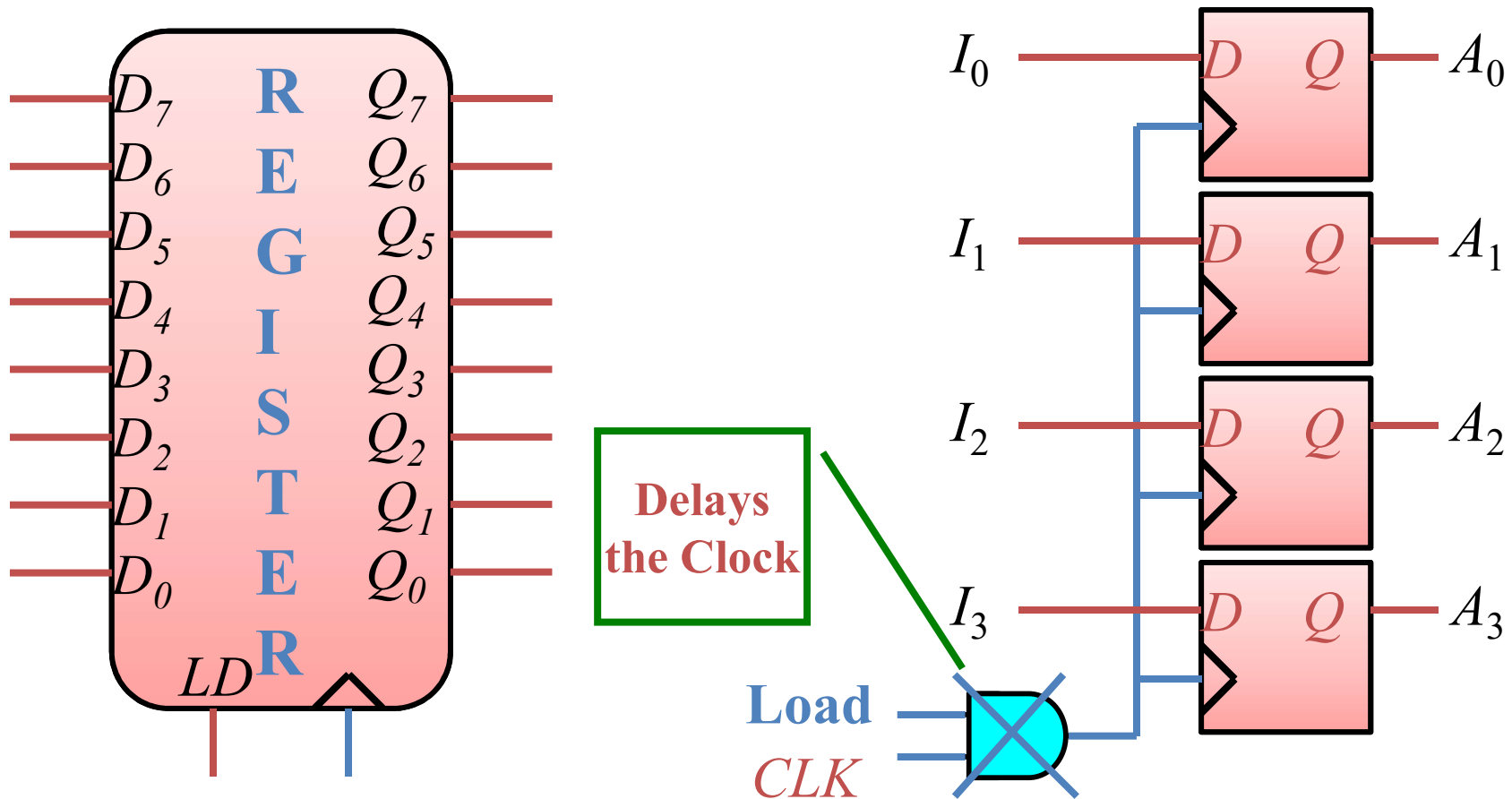
- Control **Loading** the Register with New Data



LD	$Q(t+1)$
0	$Q(t)$
1	D

Registers with Parallel Load

- Should we block the “Clock” to keep the “Data”?



Thanks